

Securing Your GitHub Project

Comprehensive Study Notes

Module	Topic
1	Course Overview
2	Safeguarding Access to Your GitHub Repository
3	Securing Your Development Workflow
4	Discovering Vulnerabilities in Dependencies
5	Publishing Security Fixes

Module 1 · Course Overview

1.1 Why GitHub Security Matters

GitHub is the world's most widely used platform for hosting and collaborating on code. Because it stores source code, secrets, CI/CD configurations, and deployment scripts, it has become a prime attack surface. A single compromised account or misconfigured repository can expose customer data, intellectual property, and production infrastructure.

1.2 The Security Mindset

Security is not a one-time task — it is a continuous practice woven into every stage of the software development lifecycle (SDLC). This course adopts a **shift-left philosophy**: finding and fixing security issues as early as possible, before code ever reaches production.

Key Principle: Shift-Left Security means detecting issues at the developer's workstation rather than after deployment. The cost of fixing a bug grows exponentially the later it is found.

1.3 Course Learning Objectives

By the end of this course you will be able to:

- Configure GitHub account security using strong authentication methods.
- Manage repository access controls and permissions appropriately.
- Integrate automated security scanning into your development workflow.
- Identify and remediate vulnerable dependencies using GitHub tooling.
- Responsibly disclose and publish security advisories and fixes.

1.4 Key Concepts Glossary

Term	Definition
Repository (Repo)	A storage location for your project's code, history, and configuration.
Branch Protection	Rules preventing direct pushes to critical branches, enforcing code review.
Secret Scanning	Automated detection of credentials or API keys accidentally committed to code.
Dependency	A third-party library or package your project relies on.
CVE	Common Vulnerabilities and Exposures — a public identifier for known security flaws.
Security Advisory	A formal notice detailing a vulnerability, its impact, and remediation steps.
SBOM	Software Bill of Materials — a complete inventory of all components in your software.

Module 2 · Safeguarding Access to Your GitHub Repository

2.1 Account Security Fundamentals

Your GitHub account is the entry point to all your repositories. Protecting it is the first and most critical step in your security posture.

2.1.1 Strong Passwords

- Use a unique, randomly generated password of at least **15 characters**.
- Never reuse passwords across services.
- Store credentials in a reputable password manager (e.g., 1Password, Bitwarden).

2.1.2 Two-Factor Authentication (2FA)

2FA adds a second layer of verification beyond your password. GitHub requires 2FA for many enterprise and open source contributions.

Method	Strength	Notes
Hardware Security Keys (FIDO2/WebAuthn)	Strongest	Physical devices like YubiKey; phishing-resistant.
Authenticator Apps (TOTP)	Strong	Apps like Google Authenticator, Authy; generates time-based codes.
SMS-based 2FA	Weak	Vulnerable to SIM-swapping; avoid if possible.

Best Practice: Always register at least two 2FA methods (e.g., a hardware key + an authenticator app) so you are not locked out if one is lost.

2.1.3 Passkeys

GitHub supports passkeys as a modern, passwordless authentication method. Passkeys use public-key cryptography and biometric authentication stored on your device, making them highly secure and resistant to phishing attacks.

2.2 Personal Access Tokens (PATs)

Personal Access Tokens replace passwords for Git operations over HTTPS and API access. They allow fine-grained control over what operations are permitted.

Feature	Fine-Grained PAT	Classic PAT
Scope	Single repository or organization	All repos you can access
Permissions	Granular per resource type	Broad permission categories
Expiry	Required (max 1 year)	Optional (can be no expiry)
Recommended	✅ Yes — principle of least privilege	Legacy use only

- Always set an expiration date on tokens.
- Grant only the minimum permissions required for the task.
- Rotate tokens regularly and immediately if a breach is suspected.
- Never commit tokens to source code — use environment variables or secret managers.

2.3 SSH Keys

SSH keys provide a secure, encrypted connection for Git operations. They are the preferred method for developers who interact with GitHub regularly.

- Generate an Ed25519 key pair: `ssh-keygen -t ed25519 -C "your_email@example.com"`
- Add the public key (`~/.ssh/id_ed25519.pub`) to GitHub under **Settings > SSH and GPG keys**.
- Use a passphrase to protect your private key.
- Never share or commit your private key.
- Audit and remove unused SSH keys regularly.

2.4 Repository Access Controls

2.4.1 Repository Visibility

Visibility	Description
Public	Anyone on the internet can view. Suitable for open source projects.
Private	Only invited collaborators can access. Use for proprietary code.
Internal (Enterprise)	Visible to all members of your enterprise. Suitable for cross-team sharing.

2.4.2 Collaborator Permission Levels

Role	Capabilities
Read	View and clone the repository; open issues and PRs.
Triage	Manage issues and PRs without write access to code.

Write	Push branches, open/merge PRs.
Maintain	Manage repository settings without access to sensitive or destructive actions.
Admin	Full access including destructive actions and security settings.

Principle of Least Privilege: Assign the lowest permission level necessary for each collaborator's role. Regularly audit and revoke access for former team members.

2.4.3 Branch Protection Rules

- Require pull request reviews before merging (set required reviewers ≥ 1).
- Require status checks to pass before merging (e.g., CI tests must be green).
- Require conversation resolution before merging.
- Require signed commits to verify authorship.
- Restrict who can push to matching branches.
- Enable **"Do not allow bypassing the above settings"** to apply rules to admins too.

2.4.4 CODEOWNERS

A **CODEOWNERS** file defines who is responsible for reviewing changes to specific files or directories. GitHub automatically requests reviews from the relevant code owners when a PR modifies those files.

- Place the file in the root, `docs/`, or `.github/` directory.
- Use glob patterns to map paths to team or individual owners.
- Example: `* @org/security-team` means all files require approval from the security team.

2.5 Audit Logs

GitHub provides audit logs that record all actions taken in an organization or repository. Regularly reviewing audit logs helps detect unauthorized access or suspicious activity.

- Access logs under **Organization Settings > Audit log**.
- Filter by action type, actor, repository, or date range.
- Export logs for SIEM integration (JSON/CSV format).
- Monitor for unusual events: unexpected role changes, new SSH keys added, mass deletions.

Module 3 · Securing Your Development Workflow

3.1 Introduction to Secure Development Practices

A secure development workflow embeds security checks at each stage of writing, reviewing, and merging code. This prevents vulnerabilities from accumulating and makes them cheaper to fix.

3.2 Secret Scanning

Secret scanning automatically detects credentials, API keys, tokens, and other sensitive strings that are accidentally committed to repositories.

3.2.1 How Secret Scanning Works

- GitHub scans every commit and PR for patterns matching known secret formats.
- When a secret is detected, GitHub alerts repository admins and, for partner patterns, notifies the secret issuer to revoke it.
- Covers **200+ secret types** from providers including AWS, Google, Stripe, and more.

3.2.2 Push Protection

Push protection extends secret scanning to **block pushes before they reach the remote**, stopping secrets at the source.

- Enable under **Settings > Code security > Secret scanning > Push protection**.
- Developers are warned at push time and can bypass with justification (which is logged).
- Use custom patterns to detect organization-specific secrets.

Critical Reminder: If a secret is ever committed — even briefly — treat it as compromised. Revoke and rotate immediately, even if you delete the commit, as history may have been cached.

3.3 Code Scanning with CodeQL

Code scanning uses static analysis to find security vulnerabilities and coding errors in your codebase. GitHub's built-in engine, **CodeQL**, treats code as data and queries it for vulnerability patterns.

3.3.1 Setting Up Code Scanning

- Go to **Security > Code scanning > Set up code scanning**.
- Choose **"Default" setup** for automatic language detection and configuration.
- Or use **"Advanced"** to add a custom GitHub Actions workflow (CodeQL YAML file).
- Scans run on push, pull request, and on a schedule.

3.3.2 Interpreting Results

- Results appear in the **Security tab** under **Code scanning alerts**.
- Each alert shows the vulnerable line, a description, and remediation advice.
- Alerts are categorized by severity: **Critical, High, Medium, Low**.
- Dismiss alerts with justification (false positive, won't fix, used in tests).

3.3.3 Common Vulnerability Categories Detected

Category	Description
SQL Injection	Unsanitized user input passed directly to database queries.
Cross-Site Scripting (XSS)	Unescaped user content rendered in HTML.
Path Traversal	User-controlled file paths allowing directory escape.
Insecure Deserialization	Unsafe object deserialization from untrusted sources.
Hardcoded Credentials	Secrets embedded directly in source code.

3.4 Security Policies (SECURITY.md)

A **SECURITY.md** file communicates how users should report vulnerabilities they discover.

- State **supported versions** of your software (which receive security patches).
- Provide clear instructions for submitting a vulnerability report privately.
- Set expectations: response time, disclosure timeline, CVE assignment process.
- GitHub surfaces this file as a link in the Security tab and issue creation flow.

3.5 Signed Commits

Commit signing uses GPG or SSH keys to cryptographically verify that a commit was authored by a specific person and was not tampered with afterward.

- Generate a GPG key and add the public key to your GitHub account.
- Configure Git to sign all commits: `git config --global commit.gpgsign true`
- Enable **"Require signed commits"** in branch protection rules.
- Signed commits display a **"Verified"** badge in GitHub's UI.

3.6 Dependency Review

The dependency review action blocks PRs that introduce dependencies with known vulnerabilities. It generates a diff of dependency changes and checks them against the GitHub Advisory Database before merging.

- Add the `dependency-review-action` to your GitHub Actions workflows.
- Configure to fail on vulnerabilities of a specified severity or higher.
- Review the generated report in the PR checks section.

Module 4 · Discovering Vulnerabilities in Dependencies

4.1 The Dependency Security Problem

Modern applications depend heavily on third-party libraries and packages. A typical JavaScript project may pull in hundreds or thousands of transitive dependencies. Each dependency is a potential security risk if it contains a vulnerability.

Key Statistic: Research consistently shows that over 75% of enterprise application codebases consist of open source components. This makes dependency security one of the highest-impact areas of application security.

4.2 The Dependency Graph

GitHub's dependency graph provides a visual map of all direct and transitive dependencies detected in your repository.

- Supports ecosystems: npm, PyPI, RubyGems, Maven, NuGet, Cargo, Composer, Go modules, and more.
- Access under **Insights > Dependency graph**.
- Shows which repos depend on your repo (used in impact analysis).
- Must be enabled in repository settings to power Dependabot alerts.

4.3 Dependabot Alerts

Dependabot alerts notify you when a dependency has a known vulnerability listed in the **GitHub Advisory Database** (which mirrors the NVD/CVE database).

4.3.1 Enabling Dependabot Alerts

- Go to **Settings > Code security > Dependabot alerts** and enable.
- GitHub scans your dependency graph on every push and when new advisories are published.
- Alerts appear in the **Security tab** under **Dependabot alerts**.

4.3.2 Reading an Alert

- Severity rating (Critical/High/Medium/Low) based on **CVSS score**.
- Affected package name, installed version, and first patched version.

- CVE identifier and link to the full advisory.
- Suggested remediation: usually upgrading to a safe version.
- Auto-dismiss options for development-only dependencies or false positives.

4.4 Dependabot Security Updates

Dependabot can automatically open pull requests to upgrade vulnerable dependencies to a patched version, reducing manual remediation work.

- Enable under **Settings > Code security > Dependabot security updates**.
- Dependabot opens a PR targeting the minimum safe version.
- The PR includes a link to the advisory and a description of the vulnerability.
- Review the PR, run your tests, and merge if everything passes.

Important: Always review and test Dependabot PRs before merging. Major version upgrades may introduce breaking changes in addition to security fixes.

4.5 Dependabot Version Updates

Beyond security patches, Dependabot can also keep all dependencies up to date with the latest releases, reducing the gap to a future patched release.

4.5.1 Configuring dependabot.yml

Create a `.github/dependabot.yml` file in your repository:

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
    reviewers:
      - "your-team"
    labels:
      - "dependencies"
```

Key	Description
package-ecosystem	The package manager (npm, pip, maven, etc.)
directory	Location of manifest file (usually /)
schedule.interval	How often to check: daily, weekly, or monthly
open-pull-requests-limit	Max open PRs Dependabot will create at once (default 5)

ignore	List of packages or versions to exclude from updates
reviewers	GitHub usernames or teams automatically requested to review PRs
labels	Labels to add to Dependabot PRs for filtering

4.6 GitHub Advisory Database

- Searchable at github.com/advisories.
- Contains severity, affected versions, patched versions, and references.
- **GitHub-reviewed advisories** are manually validated for accuracy.
- **Unreviewed advisories** come directly from the NVD feed.

4.7 Software Bill of Materials (SBOM)

An SBOM is a formal, machine-readable inventory of all software components in your application. It is increasingly required by governments and enterprises for compliance and supply chain security.

- Export from GitHub: **Insights > Dependency graph > Export SBOM**.
- Output format: **SPDX JSON** (industry standard).
- Include in releases for downstream consumers to audit your dependencies.

Module 5 · Publishing Security Fixes

5.1 The Responsible Disclosure Process

Responsible disclosure is the practice of privately reporting a vulnerability to the maintainer, giving them time to release a fix before public announcement. This protects users while giving maintainers the opportunity to patch proactively.

5.1.1 Disclosure Timeline (Best Practice)

Phase	Action
Day 0	Researcher privately reports vulnerability to maintainer.
Days 1–7	Maintainer acknowledges receipt and confirms the issue.
Days 7–90	Maintainer develops and tests a fix (90-day window is industry standard).
Day ~90	Fix is released. CVE is assigned. Advisory is published simultaneously.
After release	Full technical details may be shared with the security community.

5.2 GitHub Private Vulnerability Reporting

GitHub provides a built-in private vulnerability reporting workflow that allows security researchers to submit reports directly to maintainers without public exposure.

5.2.1 Enabling Private Reporting

- Go to **Settings > Code security > Private vulnerability reporting** and enable.
- Adds a **"Report a vulnerability"** button to your repository's Security tab.
- Reports go to repository admins and security managers only — not publicly visible.

5.2.2 Handling an Incoming Report

- Acknowledge the report promptly (within 3–7 days).
- Assess severity using CVSS scoring.
- Collaborate with the reporter in the private thread if needed.
- Accept the report to open a draft security advisory.
- Request a CVE from GitHub if one has not been assigned.

5.3 Repository Security Advisories

A repository security advisory is a private workspace where maintainers can discuss, develop a fix, and coordinate a release before publishing the advisory publicly.

5.3.1 Creating an Advisory

- Navigate to **Security > Advisories > New draft security advisory**.
- Fill in: affected ecosystem, package name, vulnerable versions, patched version.
- Add a description of the vulnerability and its impact (CVSS vector).
- Add **CWE identifiers** (Common Weakness Enumeration) to classify the flaw.

5.3.2 Private Forks for Fixes

From within a draft advisory, you can create a **temporary private fork** to collaborate on the fix without exposing the vulnerability in your public repository.

- Click **"Start a temporary private fork"** within the advisory.
- Collaborate with trusted contributors in the private fork.
- Open a pull request from the private fork to your main repo when ready.
- The fix is merged simultaneously with the advisory publication.

Security Tip: Never push security fix commits to your public main branch before publishing the advisory. Even without announcement, the commit diff reveals the vulnerability to attackers.

5.4 Requesting a CVE

A CVE identifier is a unique, publicly known reference to a vulnerability. **GitHub is a CVE Numbering Authority (CNA)**, allowing it to assign CVEs directly.

- Click **"Request CVE"** in your draft security advisory.
- GitHub reviews the request (typically within 3 business days).
- The CVE is assigned in **reserved state** until the advisory is published.
- After publication, the CVE appears in the NVD and GitHub Advisory Database.

5.5 Publishing the Advisory

5.5.1 Pre-Publication Checklist

- ☐ Fix is merged and a new patched release is available.
- ☐ Release notes for the patched version mention the security fix.
- ☐ CVE has been assigned (reserved).
- ☐ Advisory content reviewed: affected versions, severity, description.

- ☐ Coordinated with reporter on publication date.

5.5.2 What Happens on Publication

- Advisory is listed publicly on your repository's **Security Advisories** page.
- It is added to the **GitHub Advisory Database**.
- GitHub opens **Dependabot alerts** for all repositories using an affected version.
- Dependabot creates **security update PRs** automatically for affected users.
- The CVE record is updated from reserved to published in the NVD.

5.6 Communication Best Practices

- Be clear and factual in the advisory — avoid minimizing the severity.
- **Credit the reporter** if they consent to public acknowledgment.
- Link to the patched release and provide upgrade instructions.
- Use a security mailing list or GitHub Discussions to notify your community.
- Monitor for follow-up questions or related issues after publication.

Summary: Responsible fix lifecycle: receive private report → triage and develop fix in private fork → assign CVE → publish advisory and patched release simultaneously → notify your community.

Quick Reference: Key GitHub Security Settings

Setting	Where to Enable
2FA / Passkeys	GitHub Settings > Password and authentication
SSH Keys / PATs	GitHub Settings > SSH and GPG keys / Developer settings
Branch Protection Rules	Repo Settings > Branches > Add rule
CODEOWNERS	Create <code>.github/CODEOWNERS</code> in your repository
Secret Scanning + Push Protection	Repo Settings > Code security > Secret scanning
Code Scanning (CodeQL)	Repo Security > Code scanning > Set up
Dependabot Alerts	Repo Settings > Code security > Dependabot alerts
Dependabot Security Updates	Repo Settings > Code security > Dependabot security updates
Dependabot Version Updates	Create <code>.github/dependabot.yml</code>
Private Vulnerability Reporting	Repo Settings > Code security > Private vulnerability reporting
Security Advisory / CVE	Repo Security > Advisories > New draft security advisory